

Design and Evaluation of Enhanced Global History Branch Predictors for Mitigating Pattern History Table Aliasing

Jadala Chandana

Abstract

Branch prediction is critical for sustaining high performance in modern out-of-order superscalar processors. Global history predictors improve accuracy by capturing inter-branch correlations, but they suffer from Pattern History Table (PHT) aliasing, where multiple branches map to the same table entry and interfere destructively.

This work presents the design, implementation, and evaluation of two enhanced global history branch predictors implemented in the ChampSim simulator. The proposed designs improve index entropy, expand history depth, and refine confidence modeling to mitigate aliasing while analyzing hardware tradeoffs. Evaluation on SPEC CPU benchmarks demonstrates substantial MPKI reduction and measurable IPC improvements, particularly in branch-intensive workloads. Beyond raw accuracy improvement, this study analyzes architectural scaling effects and convergence behavior to better understand the tradeoff between predictor capacity and performance.

1 Problem Statement

Global history branch predictors use a Pattern History Table (PHT) indexed by combining the branch program counter (PC) and a Global History Register (GHR). When multiple dynamic branches map to the same PHT entry, one branch overwrites the learned behavior of another, causing destructive interference.

Such destructive interference increases misprediction rate and reduces speculative execution efficiency. The core challenge is therefore not merely to improve accuracy, but to reduce conflict and capacity aliasing while maintaining practical hardware cost.

The goal of this work is:

- To design branch predictors that reduce PHT aliasing.
- To maintain practical hardware cost.
- To evaluate performance relative to bimodal and gshare predictors.
- To analytically justify architectural improvements and scaling behavior.

2 Branch Prediction Fundamentals

Conditional branches introduce control hazards in deep pipelines. A misprediction results in pipeline flush, wasted speculative execution, and performance degradation proportional to pipeline depth.

Thus, branch prediction accuracy directly impacts:

- Pipeline utilization

- Speculative window depth
- Overall Instructions Per Cycle (IPC)

2.1 Global History Predictor

A global predictor consists of:

- Global History Register (GHR)
- Pattern History Table (PHT)
- Saturating counters

The GHR updates as:

$$GHR = (GHR \ll 1) | outcome$$

The GHR encodes recent control-flow patterns, enabling correlation-based prediction.

2.2 Saturating Counters

- 2-bit counters: 4 states (basic confidence model)
- 3-bit counters: 8 states (improved stability)

Higher-bit counters reduce prediction oscillation by requiring stronger contradictory evidence before flipping prediction direction.

3 Baseline: Gshare Predictor

Gshare computes:

$$Index = (PC \oplus GHR) \bmod PHT_SIZE$$

While XOR improves distribution compared to PC-only indexing, aliasing persists due to limited table capacity and history length.

When dynamic branch diversity exceeds PHT capacity, conflict aliasing becomes unavoidable. Thus, improving index mixing alone cannot eliminate destructive interference without adequate storage scaling.

4 Proposed Designs

4.1 Design 1: mybranchpred

- GHR length: 14 bits
- PHT size: 2^{14} (16K entries)
- 2-bit counters
- Threshold: ≥ 2

Enhanced hash:

$$Hash = ip \oplus (ip \gg 14) \oplus GHR$$

$$Hash = Hash \oplus (Hash \gg 16)$$

This improves entropy relative to basic gshare. However, because the PHT contains only 16K entries, different dynamic branches still frequently compete for the same index under branch-intensive workloads. Therefore, capacity aliasing remains significant.

4.2 Design 2: mybranchpred2

- GHR length: 20 bits
- PHT size: 2^{20} (1M entries)
- 3-bit counters
- Threshold: ≥ 4

Architectural improvements:

1. Longer history captures deeper branch correlation.
2. Larger PHT reduces collision probability by 64x.
3. 3-bit counters improve confidence stability.

Importantly, the increase in history length and table size is balanced. Increasing history length alone may increase index diversity beyond table capacity, potentially worsening aliasing. In mybranchpred2, proportional scaling ensures that expanded correlation modeling translates into effective accuracy gains.

5 Experimental Methodology

Simulator: ChampSim Warmup: 1M instructions Simulation windows: 20M and 140M Benchmarks: 403.gcc and 445.gobmk

Metrics:

- Accuracy (%)
- MPKI
- IPC
- ROB occupancy

MPKI is defined as:

$$MPKI = \frac{Mispredictions}{Instructions} \times 1000$$

6 Implementation Details

Both predictors were implemented inside the ChampSim framework by modifying the branch prediction module. The key implementation components include:

- Global History Register (GHR) update logic
- Pattern History Table (PHT) indexing using XOR-based hash mixing
- Saturating counter update mechanism
- Threshold-based taken/not-taken decision logic

mybranchpred2 extends mybranchpred by:

- Increasing GHR length from 14 to 20 bits
- Expanding PHT from 16K to 1M entries
- Upgrading from 2-bit to 3-bit counters
- Adjusting prediction threshold from 2 to 4

All other pipeline and simulation parameters remain identical, ensuring fair comparison.

7 Results

This section presents full experimental results for both 20M and 140M instruction windows. All evaluated predictors are included to ensure complete and transparent comparison.

7.1 403.gcc – 20M Instructions

Predictor	Accuracy	MPKI	IPC	ROB
bimodal	99.627	0.73225	—	44.5555
gshare	99.7443	0.502	—	56.564
mybranchpred	99.7298	0.53035	—	53.9266
mybranchpred2	99.8087	0.3756	—	65.9054

Observation:

- mybranchpred performs similarly to gshare, indicating that hash enhancement alone provides limited improvement under constrained table size.
- mybranchpred2 achieves noticeable MPKI reduction even in shorter execution windows.
- Increased ROB occupancy suggests improved speculative confidence.

7.2 403.gcc – 140M Instructions

Predictor	Accuracy	MPKI	IPC	ROB
bimodal	99.65	0.684	1.1874	45.38
gshare	99.78	0.42	1.1932	63.09
mybranchpred	99.78	0.431	1.2004	62.77
mybranchpred2	99.89	0.203	1.2033	114.95

Analysis:

MPKI reduction vs gshare:

$$\frac{0.42 - 0.203}{0.42} = 51.7\%$$

- Longer execution allows deeper history (20-bit GHR) to stabilize.
- Collision reduction from 1M-entry PHT becomes more effective.
- IPC improvement is modest because gcc is already highly predictable.
- Significant ROB growth reflects deeper sustained speculation.

7.3 445.gobmk – 20M Instructions

Predictor	Accuracy	MPKI	ROB
bimodal	86.18	24.74	18.5
gshare	84.32	28.06	14.79
mybranchpred	84.189	28.31	14.69
mybranchpred2	91.607	15.031	29.06

Observation:

- mybranchpred shows negligible improvement over gshare.
- This confirms that improved hashing alone does not resolve heavy aliasing.
- mybranchpred2 significantly reduces MPKI even in shorter windows.
- Large ROB increase indicates stronger speculative stability.

7.4 445.gobmk – 140M Instructions

Predictor	Accuracy	MPKI	ROB
bimodal	86.22	24.44	18.01
gshare	85.59	25.55	16.07
mybranchpred	85.53	25.67	15.98
mybranchpred2	94.47	9.8	42.85

Analysis:

MPKI reduction vs gshare:

$$\frac{25.55 - 9.8}{25.55} = 61.6\%$$

- gobmk exhibits complex long-range branch correlation.
- 20-bit history captures deeper execution patterns.
- 1M-entry PHT dramatically reduces destructive aliasing.
- Improvement increases from 20M to 140M, indicating training stability.

8 Integrated Analytical Discussion

This section synthesizes architectural design decisions with observed performance behavior across both benchmarks and execution lengths.

8.1 Correlation Modeling and History Depth

The primary architectural distinction between mybranchpred and mybranchpred2 lies in Global History Register (GHR) length. A 14-bit GHR in mybranchpred encodes only short-range correlation, limiting the predictor’s ability to distinguish execution contexts when branch behavior depends on deeper control-flow patterns.

By extending the GHR to 20 bits in mybranchpred2, the predictor expands its theoretical history space from 2^{14} to 2^{20} possible patterns. This increased context granularity enables finer differentiation between dynamic execution states.

The impact is clearly visible in gobmk, where MPKI reduces from 25.55 to 9.8 at 140M instructions. This substantial reduction confirms that the workload exhibits strong long-range branch correlation, which shorter history predictors fail to capture.

8.2 PHT Capacity and Aliasing Mitigation

While deeper history improves context representation, it is effective only when supported by adequate Pattern History Table (PHT) capacity. The PHT size increases from 16K entries in mybranchpred to 1M entries in mybranchpred2, representing a 64x expansion.

Collision probability can be approximated as:

$$P(\text{collision}) \propto \frac{N_{\text{dynamic branches}}}{\text{PHT size}}$$

Thus, enlarging the PHT substantially reduces destructive interference between unrelated branches.

The experimental data supports this: mybranchpred shows minimal improvement over gshare in gobmk, indicating that hash refinement alone cannot resolve heavy aliasing. Only when PHT capacity is scaled proportionally with history depth does prediction accuracy improve dramatically.

This demonstrates that correlation depth and storage capacity must be architecturally balanced.

8.3 Counter Granularity and Prediction Stability

The transition from 2-bit to 3-bit saturating counters further enhances stability. In 2-bit designs, limited contradictory updates can flip prediction direction quickly. In contrast, 3-bit counters (threshold ≥ 4) require stronger evidence before changing prediction polarity.

This has two important effects:

- Noise from occasional aliasing does not immediately corrupt learned state.
- Prediction oscillation between taken and not-taken is reduced.

As a result, mybranchpred2 exhibits smoother learning behavior and more stable convergence over long instruction windows.

8.4 Convergence Behavior Across Execution Lengths

The comparison between 20M and 140M instruction windows reveals important convergence characteristics.

At 20M instructions, mybranchpred2 already demonstrates significant MPKI reduction. However, improvement becomes more pronounced at 140M instructions. This indicates that deeper history predictors require longer training phases to fully stabilize learned correlations.

The performance scaling from 20M to 140M confirms that increased predictor capacity benefits from extended execution, leading to improved steady-state accuracy.

8.5 Workload-Specific Behavior: gcc vs gobmk

The two benchmarks respond differently to architectural scaling:

gcc:

- Baseline prediction accuracy is already high.
- MPKI reduction exceeds 50%.
- IPC improvement is modest (1.1932 to 1.2033).

This suggests that branch misprediction is not the dominant performance bottleneck in gcc. Memory hierarchy effects and limited instruction-level parallelism constrain achievable IPC, leading to diminishing returns from further prediction improvements.

gobmk:

- Baseline aliasing is severe.
- Prediction accuracy improves from 85.59% to 94.47%.
- MPKI reduction exceeds 60%.

This indicates strong long-range correlation and heavy aliasing pressure in smaller predictors. The architectural scaling of history depth and PHT capacity directly addresses these characteristics.

8.6 Speculative Efficiency and ROB Occupancy

Reorder Buffer (ROB) occupancy increases significantly in mybranchpred2. This behavior reflects improved speculative execution confidence:

- Fewer pipeline flushes occur due to reduced mispredictions.
- The speculative window remains filled for longer durations.
- The processor sustains higher instruction throughput.

Higher ROB utilization therefore serves as indirect evidence of improved control-flow prediction stability.

8.7 Architectural Tradeoff Perspective

The improvements in mybranchpred2 come at the cost of substantial storage increase (4KB to approximately 384KB).

This highlights a classical microarchitectural tradeoff:

- Larger predictors reduce aliasing and improve accuracy.
- Increased storage impacts silicon area and energy consumption.
- Performance benefits depend strongly on workload characteristics.

Therefore, predictor design must balance correlation depth, table capacity, counter granularity, and hardware cost to achieve optimal efficiency.

9 Hardware Cost

Predictor	Entries	Counter Bits	Storage
mybranchpred	16K	2	4 KB
mybranchpred2	1M	3	384 KB

The storage increase from 4KB to approximately 384KB represents a substantial hardware tradeoff. However, in high-performance processors where branch misprediction penalties are severe, such overhead may be justified. This reflects the classical architecture tradeoff between silicon area and performance gain.

10 Limitations and Future Work

- No tagging mechanism to fully eliminate aliasing
- Increased storage overhead
- Limited benchmark coverage

Future directions:

- Tagged predictors
- TAGE-based models
- Hybrid global-local predictors
- Skewed associative PHT designs

11 Conclusion

This study demonstrates that increasing history depth, expanding PHT capacity, and refining counter granularity significantly mitigate Pattern History Table aliasing. Experimental results show over 50% MPKI reduction in complex workloads, particularly in branch-intensive applications.

The results further illustrate that predictor scaling must be balanced: deeper correlation modeling is effective only when supported by adequate storage capacity. This work highlights

the importance of entropy-aware indexing, architectural scaling, and workload-aware evaluation in modern branch predictor design.

These findings demonstrate that branch predictor effectiveness is not solely a function of history depth, but of balanced architectural scaling between correlation modeling and storage capacity.

References

1. ChampSim Simulator Documentation. Available at:
<https://champsim.github.io/ChampSim/>
2. used for reference. Available at:
<https://course.ece.cmu.edu/~ece740/f15/lib/exe/fetch.php?media=18-740-fall15-lecture05.pdf>